

My Project

Generated by Doxygen 1.12.0

1 Project 3 - Differential Evolution & Partile Swarm Algorithms (C)	1
1.1 Overview	1
1.2 Program utilizes both Blind Search and Local Search algorithms	1
1.3 CHANGELOG:	1
1.4 4. Updated README.md	1
1.5 KNOWN ISSUES	1
1.6 1. PSO inertia weight is incorrectly adjusted.	1
1.7 Build Proceedure	1
1.8 File Structure	2
1.9 Config File (Required)	2
1.10 =====	2
1.11 REQUIRED PARAMETERS	2
1.12 =====	2
1.13 Problem dimension	2
1.14 Problem number	3
1.15 1 = Schwefel	3
1.16 2 = DeJong1	3
1.17 3 = Rosenbrock	3
1.18 4 = Rastrigin	3
1.19 ...	3
1.20 Algorithm selection	3
1.21 Options: RLS, DE, PSO	3
1.22 Output file for aggregated results	3
1.23 Bounds (lower MUST be < upper)	3
1.24 =====	3
1.25 SHARED PARAMETERS	3
1.26 =====	3
1.27 Number of generations (DE/PSO)	3
1.28 Population size (DE)	3
1.29 Swarm size (PSO)	3
1.30 =====	4
1.31 PSO PARAMETERS	4
1.32 =====	4
1.33 Inertia weight	4
1.34 Cognitive coefficient	4
1.35 Social coefficient	4
1.36 =====	4
1.37 DE PARAMETERS	4
1.38 =====	4
1.39 =====	4
1.40 OPTIONAL PARAMETERS	4
1.41 =====	4

1.42 Secondary dimension (if required by driver)	4
1.43 Seed for reproducibility	4
1.44 Use an integer value or SYS_TIME	4
2 Namespace Index	5
2.1 Namespace List	5
3 Class Index	7
3.1 Class List	7
4 File Index	9
4.1 File List	9
5 Namespace Documentation	11
5.1 run Namespace Reference	11
5.1.1 Detailed Description	11
5.1.2 Function Documentation	11
5.1.2.1 main()	11
5.1.2.2 read_cfg_lines()	12
5.1.2.3 read_run_csv()	12
5.1.2.4 run_once()	12
5.1.2.5 set_cfg_value()	12
5.1.2.6 write_cfg()	12
6 Class Documentation	13
6.1 Config Struct Reference	13
6.1.1 Detailed Description	13
6.1.2 Member Data Documentation	13
6.1.2.1 alg	13
6.1.2.2 c1	14
6.1.2.3 c2	14
6.1.2.4 CR	14
6.1.2.5 de_strategy	14
6.1.2.6 F	14
6.1.2.7 generations	14
6.1.2.8 lambda	14
6.1.2.9 lower	14
6.1.2.10 m	14
6.1.2.11 max_ls_steps	14
6.1.2.12 n	14
6.1.2.13 neighbors	14
6.1.2.14 output_csv	15
6.1.2.15 pop_size	15
6.1.2.16 problem_type	15

6.1.2.17 seed	15
6.1.2.18 step_frac	15
6.1.2.19 swarm_size	15
6.1.2.20 upper	15
6.1.2.21 w	15
6.2 Fitness Struct Reference	15
6.2.1 Detailed Description	15
6.2.2 Member Data Documentation	16
6.2.2.1 n	16
6.2.2.2 values	16
6.3 Population Struct Reference	16
6.3.1 Detailed Description	16
6.3.2 Member Data Documentation	16
6.3.2.1 data	16
6.3.2.2 m	16
6.3.2.3 n	16
6.4 Problem Struct Reference	16
6.4.1 Detailed Description	17
6.4.2 Member Data Documentation	17
6.4.2.1 type	17
7 File Documentation	19
7.1 include/algorithms.h File Reference	19
7.1.1 Detailed Description	19
7.1.2 Function Documentation	20
7.1.2.1 blind_search()	20
7.1.2.2 diff_evo()	20
7.1.2.3 local_search()	21
7.1.2.4 p_swarm()	22
7.1.2.5 repeated_local_search()	22
7.2 algorithms.h	23
7.3 include/config.h File Reference	24
7.3.1 Detailed Description	25
7.3.2 Enumeration Type Documentation	25
7.3.2.1 AlgorithmType	25
7.3.3 Function Documentation	25
7.3.3.1 config_load()	25
7.4 config.h	26
7.5 include/csv.h File Reference	26
7.5.1 Detailed Description	27
7.5.2 Function Documentation	27
7.5.2.1 csv_algorithm_name()	27

7.5.2.2 csv_append_result()	27
7.5.2.3 csv_init_results()	28
7.6 csv.h	28
7.7 include/mt19937ar.h File Reference	29
7.7.1 Detailed Description	29
7.7.2 Function Documentation	29
7.7.2.1 genrand_int32()	29
7.7.2.2 genrand_real2()	29
7.7.2.3 init_genrand()	29
7.8 mt19937ar.h	30
7.9 include/population.h File Reference	30
7.9.1 Detailed Description	31
7.9.2 Function Documentation	31
7.9.2.1 fitness_free()	31
7.9.2.2 fitness_init()	31
7.9.2.3 population_evaluate()	31
7.9.2.4 population_free()	32
7.9.2.5 population_init()	32
7.9.2.6 population_randomize()	32
7.9.2.7 population_row()	33
7.10 population.h	33
7.11 include/problem.h File Reference	34
7.11.1 Detailed Description	34
7.11.2 Enumeration Type Documentation	34
7.11.2.1 ProblemType	34
7.11.3 Function Documentation	35
7.11.3.1 problem_create()	35
7.11.3.2 problem_eval()	35
7.11.3.3 problem_name()	36
7.12 problem.h	36
7.13 include/timing.h File Reference	36
7.13.1 Detailed Description	36
7.13.2 Function Documentation	37
7.13.2.1 now_ms()	37
7.14 timing.h	37
7.15 src/algorithms.c File Reference	37
7.15.1 Detailed Description	38
7.15.2 Function Documentation	38
7.15.2.1 blind_search()	38
7.15.2.2 diff_evo()	38
7.15.2.3 p_swarm()	39
7.15.2.4 repeated_local_search()	40

7.16 src/config.c File Reference	40
7.16.1 Detailed Description	41
7.16.2 Function Documentation	41
7.16.2.1 config_load()	41
7.17 src/csv.c File Reference	41
7.17.1 Detailed Description	41
7.17.2 Function Documentation	41
7.17.2.1 csv_algorithm_name()	41
7.17.2.2 csv_append_result()	42
7.17.2.3 csv_init_results()	42
7.18 src/main.c File Reference	42
7.18.1 Detailed Description	43
7.18.2 Function Documentation	43
7.18.2.1 main()	43
7.18.2.2 print_usage()	43
7.19 src/mt19937ar.c File Reference	44
7.19.1 Detailed Description	44
7.19.2 Function Documentation	44
7.19.2.1 genrand_int32()	44
7.19.2.2 genrand_real2()	44
7.19.2.3 init_genrand()	45
7.20 src/population.c File Reference	45
7.20.1 Detailed Description	45
7.20.2 Function Documentation	45
7.20.2.1 fitness_free()	45
7.20.2.2 fitness_init()	46
7.20.2.3 population_evaluate()	46
7.20.2.4 population_free()	46
7.20.2.5 population_init()	46
7.20.2.6 population_randomize()	47
7.20.2.7 population_row()	47
7.21 src/problem.c File Reference	47
7.21.1 Detailed Description	48
7.21.2 Function Documentation	48
7.21.2.1 problem_create()	48
7.21.2.2 problem_eval()	48
7.21.2.3 problem_name()	48
7.22 src/timing.c File Reference	49
7.22.1 Detailed Description	49
7.22.2 Function Documentation	49
7.22.2.1 now_ms()	49

Chapter 1

Project 3 - Differential Evolution & Particle Swarm Algorithms (C)

1.1 Overview

This project generates a **Population** of random solution vectors and evaluates one selected benchmark objective function using Blind or Repeated Local Search. It outputs the **Fitness** values (objective values) for each experiment into a CSV file for external analytics.

1.2 Program utilizes both Blind Search and Local Search algorithms

1.3 CHANGELOG:

1. Refactored [main.c](#), [config.c](#), [algorithms.c](#), and associated headers
2. Implemented DE and PSO algorithms
- 3.

1.4 4. Updated README.md

1.5 KNOWN ISSUES

1.6 1. PSO inertia weight is incorrectly adjusted.

1.7 Build Procedure

- In our local directory, build the program using ming32-make on WIN/DOS and make on UNIX/LINUX. Example execution: `ming32-make`
`python scripts/run.py -exe ./project3.exe -config input/input.cfg -runs 30 -out data/Results_471.csv`

INDIVIDUAL EXECUTION: `./project2 input/input.cfg`

1.8 File Structure

- `main.c`
Entry point. Loads config, initializes RNG, creates `Problem/Population/Fitness`, writes CSV.
- `config.h / config.c`
Reads all runtime parameters from a config file (key=value format).
- `problem.h / problem.c`
Implements benchmark objective functions (1..10).
Input: vector $x \in \mathbb{R}^m$
Output: scalar $f(x)$ (fitness)
- `algorithms.c / .h` Implements Blind Search, Local Search, Differential Evolution, Particle Swarm Algorithm, and Repeated Local Search,
- `population.h / population.c`
Implements:
 - `Population`: $\mathbb{R}^{(n \times m)}$ matrix
 - `Fitness`: $\mathbb{R}^n$ vector
 Functions to randomize population values and evaluate fitness using `Problem`.
- `csv.h / csv.c`
Writes output CSV (`experiment, fitness, .`).
- `mt19937ar.h / mt19937ar.c`
Mersenne Twister RNG (MT19937).
- `Makefile`
Builds the project.
- `run_bench.py` (optional helper script) Runs the program multiple times, appends results to a master CSV, and performs analytics in Python.

1.9 Config File (Required)

1.10 =====

1.11 REQUIRED PARAMETERS

1.12 =====

1.13 Problem dimension

`m=30`

1.14 Problem number**1.15 1 = Schwefel****1.16 2 = DeJong1****1.17 3 = Rosenbrock****1.18 4 = Rastrigin****1.19 ...**

problem=4

1.20 Algorithm selection**1.21 Options: RLS, DE, PSO**

algorithm=PSO

1.22 Output file for aggregated results

output=data/results_project3.csv

1.23 Bounds (lower MUST be $<$ upper)

lower=-30 upper=30

1.24 =====**1.25 SHARED PARAMETERS****1.26 =====****1.27 Number of generations (DE/PSO)**

generations=1000

1.28 Population size (DE)

pop_size=200

1.29 Swarm size (PSO)

swarm_size=200

1.30 =====

1.31 PSO PARAMETERS

1.32 =====

1.33 Inertia weight

w=0.729

1.34 Cognitive coefficient

c1=0.8

1.35 Social coefficient

c2=1.2

1.36 =====

1.37 DE PARAMETERS

1.38 =====

strategy=rand_1_exp F=0.9 CR=0.6 lambda=0.8

1.39 =====

1.40 OPTIONAL PARAMETERS

1.41 =====

1.42 Secondary dimension (if required by driver)

n=30

1.43 Seed for reproducibility

1.44 Use an integer value or SYS_TIME

seed=SYS_TIME

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

run	11
---------------------------	----

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Config	Configuration parameters loaded from a config file	13
Fitness	Fitness values associated with a population	15
Population	Population of candidate solutions	16
Problem	Problem descriptor	16

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

include/ algorithms.h	Optimization algorithm interfaces	19
include/ config.h	Configuration structures and parsing interface	24
include/ csv.h	CSV output utilities for experiment results	26
include/ mt19937ar.h	Mersenne Twister MT19937 random number generator interface	29
include/ population.h	Population and fitness data structures and utilities	30
include/ problem.h	Benchmark optimization problem definitions	34
include/ timing.h	Wall-clock timing utility interface	36
src/ algorithms.c	Implementation of stochastic optimization algorithms	37
src/ config.c	Configuration file parsing and validation	40
src/ csv.c	CSV output utilities for experiment results	41
src/ main.c	Main entry point for Project 3 experiments	42
src/ mt19937ar.c	Mersenne Twister MT19937 random number generator	44
src/ population.c	Population and fitness management utilities	45
src/ problem.c	Benchmark optimization problem definitions and evaluation	47
src/ timing.c	Cross-platform wall-clock timing utilities	49

Chapter 5

Namespace Documentation

5.1 run Namespace Reference

Functions

- [read_cfg_lines](#) (cfg_path)
- [set_cfg_value](#) (lines, key, value)
- [write_cfg](#) (lines, path)
- [run_once](#) (exe, cfg_path)
- [read_run_csv](#) (path)
- [main](#) ()

5.1.1 Detailed Description

```
@file run.py
@brief Driver script for Project 2 experiments.
```

This script runs the Project 2 C executable multiple times while modifying the configuration file for each run and aggregating results.

Responsibilities are split as follows:

```
C Program Responsibilities:
- Executes a single problem
- Uses one problem dimension
- Executes one algorithm
- Writes results to a CSV file
```

```
Python Script Responsibilities:
- Loops over multiple runs
- Modifies configuration values per run
- Aggregates CSV output
- Computes fitness and runtime statistics
```

5.1.2 Function Documentation

5.1.2.1 main()

```
run.main ()
```

```
@brief Main entry point for running and aggregating experiments.
```

```
This function:
- Parses command-line arguments
- Runs the executable multiple times
- Modifies the configuration per run
- Aggregates results into a master CSV
- Computes summary statistics
```

5.1.2.2 read_cfg_lines()

```
run.read_cfg_lines (  
    cfg_path)
```

@brief Reads all lines from a configuration file.

@param *cfg_path* Path to the configuration file.

@return A list of strings representing lines in the config file.

5.1.2.3 read_run_csv()

```
run.read_run_csv (  
    path)
```

@brief Reads fitness values and runtime from a single run CSV file.

@param *path* Path to the CSV file generated by the C program.

@return A tuple containing:

- List of fitness values
- Runtime in milliseconds

5.1.2.4 run_once()

```
run.run_once (  
    exe,  
    cfg_path)
```

@brief Executes the C program once using the given configuration file.

@param *exe* Path to the executable.

@param *cfg_path* Path to the configuration file.

@throws RuntimeError if the program exits with a non-zero status.

5.1.2.5 set_cfg_value()

```
run.set_cfg_value (  
    lines,  
    key,  
    value)
```

@brief Sets or updates a key-value pair in configuration file lines.

If the key exists, its value is replaced. If it does not exist, the key-value pair is appended to the configuration.

@param *lines* List of configuration file lines.

@param *key* Configuration key to modify.

@param *value* Value to assign to the key.

@return Updated list of configuration lines.

5.1.2.6 write_cfg()

```
run.write_cfg (  
    lines,  
    path)
```

@brief Writes configuration lines to a file.

@param *lines* List of configuration lines.

@param *path* Output file path.

Chapter 6

Class Documentation

6.1 Config Struct Reference

Configuration parameters loaded from a config file.

```
#include <config.h>
```

Public Attributes

- int [m](#)
- int [n](#)
- int [problem_type](#)
- [AlgorithmType](#) [alg](#)
- [DEStrategy](#) [de_strategy](#)
- int [pop_size](#)
- double [lambda](#)
- double [F](#)
- double [CR](#)
- int [swarm_size](#)
- double [w](#)
- double [c1](#)
- double [c2](#)
- int [generations](#)
- int [neighbors](#)
- double [step_frac](#)
- int [max_ls_steps](#)
- char [output_csv](#) [256]
- [uint32_t](#) [seed](#)
- double [lower](#)
- double [upper](#)

6.1.1 Detailed Description

Configuration parameters loaded from a config file.

6.1.2 Member Data Documentation

6.1.2.1 [alg](#)

[AlgorithmType](#) [Config::alg](#)

Algorithm to execute

6.1.2.2 c1

`double Config::c1`
PSO cognitive coefficient (default 1.0)

6.1.2.3 c2

`double Config::c2`
PSO social coefficient (default 1.0)

6.1.2.4 CR

`double Config::CR`
DE crossover rate (default 0.9)

6.1.2.5 de_strategy

`DEStrategy Config::de_strategy`
DE strategy to execute (if alg=DE)

6.1.2.6 F

`double Config::F`
DE mutation factor (default 0.5)

6.1.2.7 generations

`int Config::generations`
DE generations (default 1000)

6.1.2.8 lambda

`double Config::lambda`
Lambda parameter for DE (default 0.8)

6.1.2.9 lower

`double Config::lower`
Lower bound of problem domain

6.1.2.10 m

`int Config::m`
[Problem](#) dimension (10, 20, 30)

6.1.2.11 max_ls_steps

`int Config::max_ls_steps`
Max local-search steps per restart (default 200)

6.1.2.12 n

`int Config::n`
Iterations per algorithm run (default 30)

6.1.2.13 neighbors

`int Config::neighbors`
Number of neighbors for (R)LS (default 30)

6.1.2.14 output_csv

```
char Config::output_csv[256]
```

Output CSV file path

6.1.2.15 pop_size

```
int Config::pop_size
```

[Population](#) size for DE and PSO (default 50)

6.1.2.16 problem_type

```
int Config::problem_type
```

[Problem](#) identifier (1..10, or 0 = all)

6.1.2.17 seed

```
uint32_t Config::seed
```

Random seed (0 = system time)

6.1.2.18 step_frac

```
double Config::step_frac
```

Step size as fraction of search range (default 0.05)

6.1.2.19 swarm_size

```
int Config::swarm_size
```

Swarm size for PSO (default 50)

6.1.2.20 upper

```
double Config::upper
```

Upper bound of problem domain

6.1.2.21 w

```
double Config::w
```

PSO inertia weight (default 0.5)
The documentation for this struct was generated from the following file:

- [include/config.h](#)

6.2 Fitness Struct Reference

[Fitness](#) values associated with a population.
`#include <population.h>`

Public Attributes

- `int n`
- `double * values`

6.2.1 Detailed Description

[Fitness](#) values associated with a population.

6.2.2 Member Data Documentation

6.2.2.1 n

```
int Fitness::n
```

Number of fitness values

6.2.2.2 values

```
double* Fitness::values
```

Array of fitness values

The documentation for this struct was generated from the following file:

- [include/population.h](#)

6.3 Population Struct Reference

[Population](#) of candidate solutions.
`#include <population.h>`

Public Attributes

- `int` [n](#)
- `int` [m](#)
- `double *` [data](#)

6.3.1 Detailed Description

[Population](#) of candidate solutions.
The population is stored as an n-by-m matrix in row-major order, where each row represents one solution vector.

6.3.2 Member Data Documentation

6.3.2.1 data

```
double* Population::data
```

Contiguous array of size n*m (row-major)

6.3.2.2 m

```
int Population::m
```

Dimension of each individual

6.3.2.3 n

```
int Population::n
```

Number of individuals (experiments)

The documentation for this struct was generated from the following file:

- [include/population.h](#)

6.4 Problem Struct Reference

[Problem](#) descriptor.
`#include <problem.h>`

Public Attributes

- [ProblemType](#) `type`

6.4.1 Detailed Description

[Problem](#) descriptor.

6.4.2 Member Data Documentation

6.4.2.1 type

[ProblemType](#) `Problem::type`

[Problem](#) type identifier

The documentation for this struct was generated from the following file:

- `include/problem.h`

Chapter 7

File Documentation

7.1 include/algorithms.h File Reference

Optimization algorithm interfaces.

```
#include "problem.h"
#include "config.h"
#include "timing.h"
```

Functions

- int [blind_search](#) (const [Problem](#) *p, int m, int iters, double lower, double upper, double *fitness_out, double *best_out, double *time_ms_out)
Performs blind (random) search optimization.
- int [local_search](#) (const [Problem](#) *p, int m, int neighbors, double step_frac, int max_steps, double lower, double upper, double *best_out, int *steps_used, double *eval_count_out)
Performs a single local search.
- int [repeated_local_search](#) (const [Problem](#) *p, int m, int restarts, int neighbors, double step_frac, int max_steps, double lower, double upper, double *fitness_out, double *best_out, double *time_ms_out)
Performs repeated local search with random restarts.
- double [diff_evo](#) (const [Problem](#) *p, int m, int iters, int pop_size, double F, double CR, double lambda, int generations, double lower, double upper, double *fitness_out, double *best_out, double *time_ms_out, DE-Strategy strategy)
Performs Differential Evolution optimization.
- double [p_swarm](#) (const [Problem](#) *p, int m, int iters, int swarm_size, double w, int generations, double c1, double c2, double lower, double upper, double *fitness_out, double *best_out, double *time_ms_out)
Performs Particle Swarm Optimization.

7.1.1 Detailed Description

Optimization algorithm interfaces.

Particle Swarm Optimization interface.

Differential Evolution interface.

This header declares the optimization algorithms used to evaluate benchmark problems. Each algorithm reports per-iteration fitness values, the best fitness found, and total execution time.

This header declares the differential evolution (DE) algorithm used to evaluate benchmark problems. The DE implementation reports per-iteration fitness values, the best fitness found, and total execution time.

This header declares the particle swarm optimization (PSO) algorithm used to evaluate benchmark problems. The PSO implementation reports per-iteration fitness values, the best fitness found, and total execution time.

7.1.2 Function Documentation

7.1.2.1 blind_search()

```
int blind_search (
    const Problem * p,
    int m,
    int iters,
    double lower,
    double upper,
    double * fitness_out,
    double * best_out,
    double * time_ms_out)
```

Performs blind (random) search optimization.

Random solution vectors are sampled uniformly within the given bounds and evaluated independently.

Parameters

	<i>p</i>	Pointer to the optimization problem.
	<i>m</i>	Dimension of the problem.
	<i>iters</i>	Number of random samples.
	<i>lower</i>	Lower bound for each dimension.
	<i>upper</i>	Upper bound for each dimension.
	<i>fitness_out</i>	Array of length <i>iters</i> storing fitness values.
	<i>best_out</i>	Output parameter for best fitness found.
	<i>time_ms_out</i>	Output parameter for total execution time in milliseconds.

Returns

0 on success, non-zero on error.

Random solution vectors are generated uniformly within the given bounds and evaluated. The best fitness value found is returned.

Parameters

	<i>p</i>	Pointer to the optimization problem.
	<i>m</i>	Dimension of the problem.
	<i>iters</i>	Number of random samples.
	<i>lower</i>	Lower bound for each dimension.
	<i>upper</i>	Upper bound for each dimension.
	<i>fitness_out</i>	Array to store fitness values per iteration.
	<i>best_out</i>	Output parameter for the best fitness value found.
	<i>time_ms_out</i>	Output parameter for total runtime in milliseconds.

Returns

0 on success, non-zero on error.

7.1.2.2 diff_evo()

```
double diff_evo (
    const Problem * p,
    int m,
    int iters,
    int pop_size,
    double F,
    double CR,
```

```

double lambda,
int generations,
double lower,
double upper,
double * fitness_out,
double * best_out,
double * time_ms_out,
DEStrategy strategy)

```

Performs Differential Evolution optimization.

A population of candidate solutions is evolved over multiple generations using mutation and crossover operations. The best solution is tracked across generations and returned at the end.

Parameters

<i>p</i>	Pointer to the optimization problem.
<i>m</i>	Dimension of the problem.
<i>iters</i>	Number of iterations (generations).
<i>pop_size</i>	Population size.
<i>F</i>	Mutation factor.
<i>CR</i>	Crossover probability.
<i>lambda</i>	Weighting factor for "rand-to-best" strategy.
<i>generations</i>	Number of generations to evolve.
<i>lower</i>	Lower bound for each dimension.
<i>upper</i>	Upper bound for each dimension.
<i>fitness_out</i>	Array to store best fitness per generation.
<i>best_out</i>	Output parameter for best fitness found.
<i>time_ms_out</i>	Output parameter for total runtime in milliseconds.
<i>strategy</i>	DE strategy to use for mutation and crossover.

Returns

0 on success, non-zero on error.

7.1.2.3 local_search()

```

int local_search (
    const Problem * p,
    int m,
    int neighbors,
    double step_frac,
    int max_steps,
    double lower,
    double upper,
    double * best_out,
    int * steps_used,
    double * eval_count_out)

```

Performs a single local search.

Starting from an initial solution, neighboring solutions are sampled iteratively and the search moves to an improved solution when found.

Parameters

<i>p</i>	Pointer to the optimization problem.
<i>m</i>	Dimension of the problem.
<i>neighbors</i>	Number of neighbors sampled per step.
<i>step_frac</i>	Step size as a fraction of the search range.
<i>max_steps</i>	Maximum number of local search steps.
<i>lower</i>	Lower bound for each dimension.

Parameters

	<i>upper</i>	Upper bound for each dimension.
	<i>best_out</i>	Output parameter for the best fitness found.
	<i>steps_used</i>	Output parameter for number of steps performed.
	<i>eval_count_out</i>	Output parameter for number of fitness evaluations.

Returns

0 on success, non-zero on error.

7.1.2.4 p_swarm()

```
double p_swarm (
    const Problem * p,
    int m,
    int iters,
    int swarm_size,
    double w,
    int generations,
    double c1,
    double c2,
    double upper,
    double lower,
    double * fitness_out,
    double * best_out,
    double * time_ms_out)
```

Performs Particle Swarm Optimization.

A swarm of particles is evolved over multiple generations, where each particle updates its velocity and position based on its own best position and the global best position found by the swarm.

Parameters

	<i>p</i>	Pointer to the optimization problem.
	<i>m</i>	Dimension of the problem.
	<i>iters</i>	Number of iterations (generations).
	<i>swarm_size</i>	Number of particles in the swarm.
	<i>w</i>	Inertia weight for velocity update.
	<i>generations</i>	Number of generations to evolve.
	<i>c1</i>	Cognitive coefficient (personal best influence).
	<i>c2</i>	Social coefficient (global best influence).
	<i>lower</i>	Lower bound for each dimension.
	<i>upper</i>	Upper bound for each dimension.
	<i>fitness_out</i>	Array to store best fitness per generation.
	<i>best_out</i>	Output parameter for best fitness found.
	<i>time_ms_out</i>	Output parameter for total runtime in milliseconds.

Returns

0 on success, non-zero on error.

7.1.2.5 repeated_local_search()

```
int repeated_local_search (
    const Problem * p,
    int m,
    int restarts,
```

```

    int neighbors,
    double step_frac,
    int max_steps,
    double lower,
    double upper,
    double * fitness_out,
    double * best_out,
    double * time_ms_out)

```

Performs repeated local search with random restarts.

The local search algorithm is executed multiple times from randomly generated starting points. The best result across all restarts is reported.

Parameters

	<i>p</i>	Pointer to the optimization problem.
	<i>m</i>	Dimension of the problem.
	<i>restarts</i>	Number of random restarts.
	<i>neighbors</i>	Number of neighbors sampled per step.
	<i>step_frac</i>	Step size as a fraction of the search range.
	<i>max_steps</i>	Maximum local search steps per restart.
	<i>lower</i>	Lower bound for each dimension.
	<i>upper</i>	Upper bound for each dimension.
	<i>fitness_out</i>	Array of length <i>restarts</i> storing per-run fitness values.
	<i>best_out</i>	Output parameter for best fitness found.
	<i>time_ms_out</i>	Output parameter for total execution time in milliseconds.

Returns

0 on success, non-zero on error.

Each restart begins from a randomly generated solution vector, followed by a local search. The best solution across all restarts is reported.

Parameters

	<i>p</i>	Pointer to the optimization problem.
	<i>m</i>	Dimension of the problem.
	<i>restarts</i>	Number of random restarts.
	<i>neighbors</i>	Number of neighbors sampled per local step.
	<i>step_frac</i>	Step size as a fraction of the search range.
	<i>max_steps</i>	Maximum local search steps per restart.
	<i>lower</i>	Lower bound for each dimension.
	<i>upper</i>	Upper bound for each dimension.
	<i>fitness_out</i>	Array to store best fitness per restart.
	<i>best_out</i>	Output parameter for best overall fitness.
	<i>time_ms_out</i>	Output parameter for runtime in milliseconds.

Returns

0 on success, non-zero on error.

7.2 algorithms.h

[Go to the documentation of this file.](#)

```

00001 #ifndef ALGORITHMS_H
00002 #define ALGORITHMS_H
00003
00004 #include "problem.h"
00005 #include "config.h"

```

```

00006 #include "timing.h"
00007
00033 int blind_search(const Problem* p,
00034                 int m,
00035                 int iters,
00036                 double lower,
00037                 double upper,
00038                 double* fitness_out,
00039                 double* best_out,
00040                 double* time_ms_out);
00041
00060 int local_search(const Problem* p,
00061                 int m,
00062                 int neighbors,
00063                 double step_frac,
00064                 int max_steps,
00065                 double lower,
00066                 double upper,
00067                 double* best_out,
00068                 int* steps_used,
00069                 double* eval_count_out);
00070
00091 int repeated_local_search(const Problem* p,
00092                           int m,
00093                           int restarts,
00094                           int neighbors,
00095                           double step_frac,
00096                           int max_steps,
00097                           double lower,
00098                           double upper,
00099                           double* fitness_out,
00100                           double* best_out,
00101                           double* time_ms_out);
00102
00103
00113 double diff_evo(const Problem* p, int m, int iters, int pop_size, double F, double CR, double lambda,
00114                int generations,
00115                double lower, double upper, double* fitness_out, double* best_out,
00116                double* time_ms_out, DEStrategy strategy);
00117
00126 double p_swarm(const Problem* p, int m, int iters, int swarm_size, double w, int generations,
00127                double c1, double c2, double lower, double upper,
00128                double* fitness_out, double* best_out, double* time_ms_out);
00129
00130 #endif // ALGORITHMS_H

```

7.3 include/config.h File Reference

Configuration structures and parsing interface.

```
#include <stdint.h>
```

Classes

- struct [Config](#)
Configuration parameters loaded from a config file.

Enumerations

- enum [AlgorithmType](#) {
[ALG_BLIND](#) = 1 , [ALG_LOCAL](#) = 2 , [ALG_RLS](#) = 3 , [ALG_PSO](#) = 4 ,
[ALG_DE](#) = 5 , [ALG_ALL](#) = 6 }
Enumeration of supported algorithm types.
- enum [DEStrategy](#) {
[DE_RAND_1_BIN](#) = 1 , [DE_BEST_1_BIN](#) , [DE_RAND_2_BIN](#) , [DE_BEST_2_BIN](#) ,
[DE_RAND_TO_BEST_1_BIN](#) , [DE_CURRENT_TO_BEST_1](#) , [DE_CURRENT_TO_RAND_1](#) , [DE_RAND_1_TO_BEST_1](#) ,
[DE_RAND_TO_BEST_1_EXP](#) , [DE_BEST_2_EXP](#) , [DE_RAND_1_EXP](#) , [DE_BEST_1_EXP](#) ,
[DE_RAND_2_EXP](#) }

Functions

- int [config_load](#) (const char *path, [Config](#) *out_cfg)

Loads configuration values from a key-value file.

7.3.1 Detailed Description

Configuration structures and parsing interface.

This header defines the configuration options used to control optimization experiments and declares the configuration file loading function.

7.3.2 Enumeration Type Documentation

7.3.2.1 AlgorithmType

enum [AlgorithmType](#)

Enumeration of supported algorithm types.

Enumerator

ALG_BLIND	Blind (random) search
ALG_LOCAL	Single local search
ALG_RLS	Repeated local search
ALG_PSO	Particle Swarm Optimization
ALG_DE	Differential Evolution
ALG_ALL	Run all algorithms

7.3.3 Function Documentation

7.3.3.1 config_load()

```
int config_load (
    const char * path,
    Config * out_cfg)
```

Loads configuration values from a key-value file.

The configuration file should contain entries of the form:

key=value

Parameters

<i>path</i>	Path to the configuration file.
<i>out_cfg</i>	Pointer to Config structure to populate.

Returns

0 on success, non-zero on failure.

Loads configuration values from a key-value file.

The configuration file is expected to contain key-value pairs in the form:

key=value

Lines starting with '#' are treated as comments. Missing or invalid values are replaced with safe defaults.

Parameters

<i>path</i>	Path to the configuration file.
<i>out_cfg</i>	Pointer to Config structure to populate.

Returns

0 on success, 1 on invalid arguments, 2 if file cannot be opened, 3 on invalid configuration values.

7.4 config.h

[Go to the documentation of this file.](#)

```

00001 #ifndef CONFIG_H
00002 #define CONFIG_H
00003
00004 #include <stdint.h>
00005
00018 typedef enum {
00019     ALG_BLIND = 1,
00020     ALG_LOCAL = 2,
00021     ALG_RLS = 3,
00022     ALG_PSO = 4,
00023     ALG_DE = 5,
00024     ALG_ALL = 6
00025 } AlgorithmType;
00026
00027
00028 typedef enum {
00029     DE_RAND_1_BIN = 1,
00030     DE_BEST_1_BIN,
00031     DE_RAND_2_BIN,
00032     DE_BEST_2_BIN,
00033     DE_RAND_TO_BEST_1_BIN,
00034     DE_CURRENT_TO_BEST_1,
00035     DE_CURRENT_TO_RAND_1,
00036     DE_RAND_TO_BEST_1,
00037     DE_RAND_TO_BEST_1_EXP,
00038     DE_BEST_2_EXP,
00039     DE_RAND_1_EXP,
00040     DE_BEST_1_EXP,
00041     DE_RAND_2_EXP
00042 } DEStrategy;
00043
00047 typedef struct {
00048     int m;
00049     int n;
00050     int problem_type;
00051     AlgorithmType alg;
00052     DEStrategy de_strategy;
00053     int pop_size;
00054     double lambda;
00055     double F;
00056     double CR;
00057     int swarm_size;
00058     double w;
00059     double c1;
00060     double c2;
00061     int generations;
00062     int neighbors;
00063     double step_frac;
00064     int max_ls_steps;
00065     char output_csv[256];
00066     uint32_t seed;
00067     double lower;
00068     double upper;
00069 } Config;
00070
00083 int config_load(const char* path, Config* out_cfg);
00084
00085 #endif /* CONFIG_H */

```

7.5 include/csv.h File Reference

CSV output utilities for experiment results.

```

#include "config.h"
#include "problem.h"

```

Functions

- int [csv_init_results](#) (const char *path)
Initializes a CSV results file.
- int [csv_append_result](#) (const char *path, [AlgorithmType](#) alg, [ProblemType](#) problem, int m, int iteration, double fitness, double time_ms)
Appends a single result row to the CSV file.
- const char * [csv_algorithm_name](#) ([AlgorithmType](#) alg)

Returns a human-readable algorithm name.

7.5.1 Detailed Description

CSV output utilities for experiment results.

This header declares functions for creating and appending CSV files used to store optimization experiment results.

7.5.2 Function Documentation

7.5.2.1 csv_algorithm_name()

```
const char * csv_algorithm_name (
    AlgorithmType alg)
```

Returns a human-readable algorithm name.

Parameters

<i>alg</i>	Algorithm identifier.
------------	-----------------------

Returns

String name of the algorithm.

Parameters

<i>alg</i>	Algorithm type identifier.
------------	----------------------------

Returns

String representation of the algorithm.

7.5.2.2 csv_append_result()

```
int csv_append_result (
    const char * path,
    AlgorithmType alg,
    ProblemType problem,
    int m,
    int iteration,
    double fitness,
    double time_ms)
```

Appends a single result row to the CSV file.

The output row format is: algorithm, problem, dimension, iteration, fitness, time_ms

Parameters

<i>path</i>	Path to the CSV output file.
<i>alg</i>	Algorithm identifier.
<i>problem</i>	Problem identifier.
<i>m</i>	Problem dimension.
<i>iteration</i>	Iteration or restart index.
<i>fitness</i>	Fitness value.
<i>time_ms</i>	Runtime in milliseconds.

Returns

0 on success, non-zero on failure.

Appends a single result row to the CSV file.

Parameters

<code>path</code>	Path to the CSV output file.
<code>alg</code>	Algorithm identifier.
<code>problem</code>	Problem identifier.
<code>m</code>	Problem dimension.
<code>iteration</code>	Iteration or restart index.
<code>fitness</code>	Fitness value.
<code>time_ms</code>	Runtime in milliseconds.

Returns

0 on success, 1 if path is invalid, 2 if the file cannot be opened.

7.5.2.3 csv_init_results()

```
int csv_init_results (
    const char * path)
```

Initializes a CSV results file.

Creates (or overwrites) a CSV file and writes the standard header used for Project 2 results.

Parameters

<code>path</code>	Path to the CSV output file.
-------------------	------------------------------

Returns

0 on success, non-zero on failure.

Writes the CSV header row. Any existing file at the given path will be overwritten.

Parameters

<code>path</code>	Path to the CSV output file.
-------------------	------------------------------

Returns

0 on success, 1 if path is invalid, 2 if the file cannot be created.

7.6 csv.h

[Go to the documentation of this file.](#)

```
00001 #ifndef CSV_H
00002 #define CSV_H
00003
00004 #include "config.h"
00005 #include "problem.h"
00006
00024 int csv_init_results(const char* path);
00025
00041 int csv_append_result(const char* path,
00042                      AlgorithmType alg,
00043                      ProblemType problem,
00044                      int m,
00045                      int iteration,
00046                      double fitness,
00047                      double time_ms);
00048
00055 const char* csv_algorithm_name(AlgorithmType alg);
00056
00057 #endif /* CSV_H */
```

7.7 include/mt19937ar.h File Reference

Mersenne Twister MT19937 random number generator interface.

```
#include <stdint.h>
```

Functions

- void [init_genrand](#) (uint32_t s)
Initializes the random number generator with a seed.
- uint32_t [genrand_int32](#) (void)
Generates a 32-bit unsigned random integer.
- double [genrand_real2](#) (void)
Generates a floating-point random number in the range [0, 1).

7.7.1 Detailed Description

Mersenne Twister MT19937 random number generator interface.

This header declares functions for initializing and using the MT19937 pseudorandom number generator.

7.7.2 Function Documentation

7.7.2.1 [genrand_int32\(\)](#)

```
uint32_t genrand_int32 (  
    void )
```

Generates a 32-bit unsigned random integer.

Returns

Random 32-bit unsigned integer.

If the generator state has been exhausted, the state is regenerated automatically.

Returns

Random 32-bit unsigned integer.

7.7.2.2 [genrand_real2\(\)](#)

```
double genrand_real2 (  
    void )
```

Generates a floating-point random number in the range [0, 1).

Returns

Random double in the range [0,1).

Generates a floating-point random number in the range [0, 1).

Uses a 32-bit integer random value scaled to the unit interval.

Returns

Random double in the range [0,1).

7.7.2.3 [init_genrand\(\)](#)

```
void init_genrand (  
    uint32_t s)
```

Initializes the random number generator with a seed.

Parameters

s	Seed value.
---	-------------

Initializes the random number generator with a seed.

This function must be called before using the random number generator unless a default seed is acceptable.

Parameters

s	Seed value.
---	-------------

7.8 mt19937ar.h

[Go to the documentation of this file.](#)

```
00001 #ifndef MT19937AR_H
00002 #define MT19937AR_H
00003
00004 #include <stdint.h>
00005
00019 void init_genrand(uint32_t s);
00020
00026 uint32_t genrand_int32(void);
00027
00033 double genrand_real2(void);
00034
00035 #endif /* MT19937AR_H */
```

7.9 include/population.h File Reference

[Population](#) and fitness data structures and utilities.

```
#include "problem.h"
```

Classes

- struct [Population](#)
Population of candidate solutions.
- struct [Fitness](#)
Fitness values associated with a population.

Functions

- int [population_init](#) ([Population](#) *pop, int n, int m)
Initializes a population structure.
- void [population_free](#) ([Population](#) *pop)
Frees memory associated with a population.
- int [fitness_init](#) ([Fitness](#) *fit, int n)
Initializes a fitness structure.
- void [fitness_free](#) ([Fitness](#) *fit)
Frees memory associated with a fitness structure.
- void [population_randomize](#) ([Population](#) *pop, double lower, double upper)
Fills a population with uniformly random values.
- void [population_evaluate](#) (const [Population](#) *pop, const [Problem](#) *prob, [Fitness](#) *fit)
Evaluates the fitness of each individual in the population.
- const double * [population_row](#) (const [Population](#) *pop, int i)
Returns a pointer to a specific population row.

7.9.1 Detailed Description

[Population](#) and fitness data structures and utilities.

This header defines structures and functions for managing populations of candidate solutions and their associated fitness values.

7.9.2 Function Documentation

7.9.2.1 `fitness_free()`

```
void fitness_free (
    Fitness * fit)
```

Frees memory associated with a fitness structure.

Parameters

<code>fit</code>	Pointer to Fitness structure.
------------------	---

7.9.2.2 `fitness_init()`

```
int fitness_init (
    Fitness * fit,
    int n)
```

Initializes a fitness structure.

Allocates memory for storing fitness values.

Parameters

<code>fit</code>	Pointer to Fitness structure.
<code>n</code>	Number of fitness values.

Returns

0 on success, non-zero on failure.

Parameters

<code>fit</code>	Pointer to Fitness structure.
<code>n</code>	Number of fitness values.

Returns

0 on success, 1 on invalid arguments, 2 on allocation failure.

7.9.2.3 `population_evaluate()`

```
void population_evaluate (
    const Population * pop,
    const Problem * prob,
    Fitness * fit)
```

Evaluates the fitness of each individual in the population.

[Fitness](#) values are written to the provided [Fitness](#) structure.

Parameters

<code>pop</code>	Pointer to Population structure.
<code>prob</code>	Pointer to the optimization problem.
<code>fit</code>	Pointer to Fitness structure.

[Fitness](#) values are stored in the corresponding [Fitness](#) structure.

Parameters

<i>pop</i>	Pointer to Population structure.
<i>prob</i>	Pointer to the optimization problem.
<i>fit</i>	Pointer to Fitness structure.

7.9.2.4 population_free()

```
void population_free (
    Population * pop)
```

Frees memory associated with a population.

Parameters

<i>pop</i>	Pointer to Population structure.
------------	--

7.9.2.5 population_init()

```
int population_init (
    Population * pop,
    int n,
    int m)
```

Initializes a population structure.

Allocates memory for an n-by-m population matrix.

Parameters

<i>pop</i>	Pointer to Population structure.
<i>n</i>	Number of individuals.
<i>m</i>	Dimension of each individual.

Returns

0 on success, non-zero on failure.

Allocates memory for an n-by-m matrix where each row represents a candidate solution vector.

Parameters

<i>pop</i>	Pointer to Population structure.
<i>n</i>	Number of individuals.
<i>m</i>	Dimension of each individual.

Returns

0 on success, 1 on invalid arguments, 2 on allocation failure.

7.9.2.6 population_randomize()

```
void population_randomize (
    Population * pop,
    double lower,
    double upper)
```

Fills a population with uniformly random values.

Values are generated within the specified lower and upper bounds.

Parameters

<i>pop</i>	Pointer to Population structure.
<i>lower</i>	Lower bound for values.
<i>upper</i>	Upper bound for values.

Fills a population with uniformly random values.

Each element of the population is assigned a random value uniformly distributed within the given bounds.

Parameters

<i>pop</i>	Pointer to Population structure.
<i>lower</i>	Lower bound for values.
<i>upper</i>	Upper bound for values.

7.9.2.7 population_row()

```
const double * population_row (
    const Population * pop,
    int i)
```

Returns a pointer to a specific population row.

Parameters

<i>pop</i>	Pointer to Population structure.
<i>i</i>	Row index.

Returns

Pointer to the requested solution vector.

Returns a pointer to a specific population row.

Parameters

<i>pop</i>	Pointer to Population structure.
<i>i</i>	Row index.

Returns

Pointer to the requested population row.

7.10 population.h

[Go to the documentation of this file.](#)

```
00001 #ifndef POPULATION_H
00002 #define POPULATION_H
00003
00004 #include "problem.h"
00005
00021 typedef struct {
00022     int n;
00023     int m;
00024     double* data;
00025 } Population;
00026
00030 typedef struct {
00031     int n;
00032     double* values;
00033 } Fitness;
00034
00045 int population\_init(Population* pop, int n, int m);
00046
```

```

00052 void population_free(Population* pop);
00053
00063 int fitness_init(Fitness* fit, int n);
00064
00070 void fitness_free(Fitness* fit);
00071
00081 void population_randomize(Population* pop, double lower, double upper);
00082
00092 void population_evaluate(const Population* pop, const Problem* prob, Fitness* fit);
00093
00101 const double* population_row(const Population* pop, int i);
00102
00103 #endif /* POPULATION_H */

```

7.11 include/problem.h File Reference

Benchmark optimization problem definitions.

Classes

- struct [Problem](#)
Problem descriptor.

Enumerations

- enum [ProblemType](#) {
[PROB_SCHWEFEL](#) = 1, [PROB_DEJONG1](#), [PROB_ROSENBROCK](#), [PROB_RASTRIGIN](#),
[PROB_GRIEWANGK](#), [PROB_SINE_ENV_SINE_WAVE](#), [PROB_STRETCH_V_SINE_WAVE](#), [PROB_ACKLEY_ONE](#),
[PROB_ACKLEY_TWO](#), [PROB_EGG HOLDER](#) }
Enumeration of supported benchmark problem types.

Functions

- [Problem](#) [problem_create](#) ([ProblemType](#) t)
Creates a [Problem](#) structure.
- const char * [problem_name](#) (const [Problem](#) *p)
Returns a human-readable name for a problem.
- double [problem_eval](#) (const [Problem](#) *p, const double *x, int m)
Evaluates the objective function for a given problem.

7.11.1 Detailed Description

Benchmark optimization problem definitions.

This header defines the problem types used in the project and declares functions for creating, naming, and evaluating optimization benchmark problems.

7.11.2 Enumeration Type Documentation

7.11.2.1 ProblemType

```
enum ProblemType
```

Enumeration of supported benchmark problem types.

Enumerator

PROB_SCHWEFEL	Schwefel function
PROB_DEJONG1	De Jong (Sphere) function
PROB_ROSENBROCK	Rosenbrock function
PROB_RASTRIGIN	Rastrigin function
PROB_GRIEWANGK	Griewangk function

Enumerator

	PROB_SINE_ENV_SINE_WAVE	Sine Envelope Sine Wave function
	PROB_STRETCH_V_SINE_WAVE	Stretch V Sine Wave function
	PROB_ACKLEY_ONE	Ackley function (variant one)
	PROB_ACKLEY_TWO	Ackley function (variant two)
	PROB_EGG HOLDER	Egg Holder function

7.11.3 Function Documentation

7.11.3.1 problem_create()

```
Problem problem_create (
    ProblemType t)
```

Creates a [Problem](#) structure.

Parameters

<i>t</i>	Problem type identifier.
----------	--

Returns

Initialized [Problem](#) structure.

Creates a [Problem](#) structure.

Parameters

<i>t</i>	Problem type identifier.
----------	--

Returns

Initialized [Problem](#) structure.

7.11.3.2 problem_eval()

```
double problem_eval (
    const Problem * p,
    const double * x,
    int m)
```

Evaluates the objective function for a given problem.

Parameters

<i>p</i>	Pointer to the problem definition.
<i>x</i>	Input solution vector.
<i>m</i>	Dimension of the solution vector.

Returns

[Fitness](#) value.

Evaluates the objective function for a given problem.

Parameters

<i>p</i>	Pointer to the problem definition.
<i>x</i>	Input solution vector.
<i>m</i>	Dimension of the solution vector.

Returns

[Fitness](#) value, or NAN if inputs are invalid.

7.11.3.3 problem_name()

```
const char * problem_name (
    const Problem * p)
```

Returns a human-readable name for a problem.

Parameters

<code>p</code>	Pointer to the problem.
----------------	-------------------------

Returns

String name of the problem.

7.12 problem.h

[Go to the documentation of this file.](#)

```
00001 #ifndef PROBLEM_H
00002 #define PROBLEM_H
00003
00016 typedef enum {
00017     PROB_SCHWEFEL = 1,
00018     PROB_DEJONG1,
00019     PROB_ROSENBROCK,
00020     PROB_RASTRIGIN,
00021     PROB_GRIEWANGK,
00022     PROB_SINE_ENV_SINE_WAVE,
00023     PROB_STRETCH_V_SINE_WAVE,
00024     PROB_ACKLEY_ONE,
00025     PROB_ACKLEY_TWO,
00026     PROB_EGG HOLDER
00027 } ProblemType;
00028
00032 typedef struct {
00033     ProblemType type;
00034 } Problem;
00035
00042 Problem problem_create(ProblemType t);
00043
00050 const char* problem_name(const Problem* p);
00051
00060 double problem_eval(const Problem* p, const double* x, int m);
00061
00062 #endif // PROBLEM_H
```

7.13 include/timing.h File Reference

Wall-clock timing utility interface.

Functions

- double [now_ms](#) (void)
Returns the current wall-clock time in milliseconds.

7.13.1 Detailed Description

Wall-clock timing utility interface.

This header declares a portable function for obtaining the current wall-clock time in milliseconds.

7.13.2 Function Documentation

7.13.2.1 now_ms()

```
double now_ms (
    void )
```

Returns the current wall-clock time in milliseconds.

The underlying implementation is platform-specific and uses high-resolution timers when available.

Returns

Current time in milliseconds.

Returns the current wall-clock time in milliseconds.

Uses CLOCK_MONOTONIC when available, otherwise falls back to clock().

Returns

Current time in milliseconds.

7.14 timing.h

[Go to the documentation of this file.](#)

```
00001 #ifndef TIMING_H
00002 #define TIMING_H
00003
00020 double now_ms(void);
00021
00022 #endif /* TIMING_H */
```

7.15 src/algorithms.c File Reference

Implementation of stochastic optimization algorithms.

```
#include "algorithms.h"
#include "mt19937ar.h"
#include "timing.h"
#include "config.h"
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <stdio.h>
```

Functions

- int [blind_search](#) (const [Problem](#) *p, int m, int iters, double lower, double upper, double *fitness_out, double *best_out, double *time_ms_out)
Performs blind (random) search optimization.
- int [repeated_local_search](#) (const [Problem](#) *p, int m, int restarts, int neighbors, double step_frac, int max_steps, double lower, double upper, double *fitness_out, double *best_out, double *time_ms_out)
Performs repeated local search with random restarts.
- double [diff_evo](#) (const [Problem](#) *p, int m, int iters, int pop_size, double F, double CR, double lambda, int generations, double lower, double upper, double *fitness_out, double *best_out, double *time_ms_out, DE-Strategy strategy)
Performs Differential Evolution optimization.
- double [p_swarm](#) (const [Problem](#) *p, int m, int iters, int swarm_size, double w, int generations, double c1, double c2, double lower, double upper, double *fitness_out, double *best_out, double *time_ms_out)
Performs Particle Swarm Optimization.

7.15.1 Detailed Description

Implementation of stochastic optimization algorithms.

This file contains implementations of blind search and repeated local search algorithms used to evaluate optimization problems. Timing measurements exclude configuration and file I/O.

7.15.2 Function Documentation

7.15.2.1 blind_search()

```
int blind_search (
    const Problem * p,
    int m,
    int iters,
    double lower,
    double upper,
    double * fitness_out,
    double * best_out,
    double * time_ms_out)
```

Performs blind (random) search optimization.

Random solution vectors are generated uniformly within the given bounds and evaluated. The best fitness value found is returned.

Parameters

	<i>p</i>	Pointer to the optimization problem.
	<i>m</i>	Dimension of the problem.
	<i>iters</i>	Number of random samples.
	<i>lower</i>	Lower bound for each dimension.
	<i>upper</i>	Upper bound for each dimension.
	<i>fitness_out</i>	Array to store fitness values per iteration.
	<i>best_out</i>	Output parameter for the best fitness value found.
	<i>time_ms_out</i>	Output parameter for total runtime in milliseconds.

Returns

0 on success, non-zero on error.

7.15.2.2 diff_evo()

```
double diff_evo (
    const Problem * p,
    int m,
    int iters,
    int pop_size,
    double F,
    double CR,
    double lambda,
    int generations,
    double lower,
    double upper,
    double * fitness_out,
    double * best_out,
    double * time_ms_out,
    DEStrategy strategy)
```

Performs Differential Evolution optimization.

A population of candidate solutions is evolved over multiple generations using mutation and crossover operations. The best solution is tracked across generations and returned at the end.

Parameters

<i>p</i>	Pointer to the optimization problem.
<i>m</i>	Dimension of the problem.
<i>iters</i>	Number of iterations (generations).
<i>pop_size</i>	Population size.
<i>F</i>	Mutation factor.
<i>CR</i>	Crossover probability.
<i>lambda</i>	Weighting factor for "rand-to-best" strategy.
<i>generations</i>	Number of generations to evolve.
<i>lower</i>	Lower bound for each dimension.
<i>upper</i>	Upper bound for each dimension.
<i>fitness_out</i>	Array to store best fitness per generation.
<i>best_out</i>	Output parameter for best fitness found.
<i>time_ms_out</i>	Output parameter for total runtime in milliseconds.
<i>strategy</i>	DE strategy to use for mutation and crossover.

Returns

0 on success, non-zero on error.

7.15.2.3 p_swarm()

```
double p_swarm (
    const Problem * p,
    int m,
    int iters,
    int swarm_size,
    double w,
    int generations,
    double c1,
    double c2,
    double upper,
    double lower,
    double * fitness_out,
    double * best_out,
    double * time_ms_out)
```

Performs Particle Swarm Optimization.

A swarm of particles is evolved over multiple generations, where each particle updates its velocity and position based on its own best position and the global best position found by the swarm.

Parameters

<i>p</i>	Pointer to the optimization problem.
<i>m</i>	Dimension of the problem.
<i>iters</i>	Number of iterations (generations).
<i>swarm_size</i>	Number of particles in the swarm.
<i>w</i>	Inertia weight for velocity update.
<i>generations</i>	Number of generations to evolve.
<i>c1</i>	Cognitive coefficient (personal best influence).
<i>c2</i>	Social coefficient (global best influence).
<i>lower</i>	Lower bound for each dimension.
<i>upper</i>	Upper bound for each dimension.
<i>fitness_out</i>	Array to store best fitness per generation.
<i>best_out</i>	Output parameter for best fitness found.
<i>time_ms_out</i>	Output parameter for total runtime in milliseconds.

Returns

0 on success, non-zero on error.

7.15.2.4 repeated_local_search()

```
int repeated_local_search (
    const Problem * p,
    int m,
    int restarts,
    int neighbors,
    double step_frac,
    int max_steps,
    double lower,
    double upper,
    double * fitness_out,
    double * best_out,
    double * time_ms_out)
```

Performs repeated local search with random restarts.

Each restart begins from a randomly generated solution vector, followed by a local search. The best solution across all restarts is reported.

Parameters

<i>p</i>	Pointer to the optimization problem.
<i>m</i>	Dimension of the problem.
<i>restarts</i>	Number of random restarts.
<i>neighbors</i>	Number of neighbors sampled per local step.
<i>step_frac</i>	Step size as a fraction of the search range.
<i>max_steps</i>	Maximum local search steps per restart.
<i>lower</i>	Lower bound for each dimension.
<i>upper</i>	Upper bound for each dimension.
<i>fitness_out</i>	Array to store best fitness per restart.
<i>best_out</i>	Output parameter for best overall fitness.
<i>time_ms_out</i>	Output parameter for runtime in milliseconds.

Returns

0 on success, non-zero on error.

7.16 src/config.c File Reference

Configuration file parsing and validation.

```
#include "config.h"
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>
#include <time.h>
```

Functions

- int [config_load](#) (const char *path, [Config](#) *out_cfg)

Loads configuration values from a file.

7.16.1 Detailed Description

Configuration file parsing and validation.

This module loads key-value configuration settings from a text file into a [Config](#) structure. The configuration controls problem dimensions, algorithm selection, runtime parameters, random seed, and output paths.

7.16.2 Function Documentation

7.16.2.1 config_load()

```
int config_load (
    const char * path,
    Config * out_cfg)
```

Loads configuration values from a file.

Loads configuration values from a key-value file.

The configuration file is expected to contain key-value pairs in the form:

key=value

Lines starting with '#' are treated as comments. Missing or invalid values are replaced with safe defaults.

Parameters

<code>path</code>	Path to the configuration file.
<code>out_cfg</code>	Pointer to Config structure to populate.

Returns

0 on success, 1 on invalid arguments, 2 if file cannot be opened, 3 on invalid configuration values.

7.17 src/csv.c File Reference

CSV output utilities for experiment results.

```
#include "csv.h"
#include <stdio.h>
#include <string.h>
```

Functions

- `const char * csv_algorithm_name (AlgorithmType alg)`
Returns a human-readable algorithm name.
- `int csv_init_results (const char *path)`
Initializes a CSV results file.
- `int csv_append_result (const char *path, AlgorithmType alg, ProblemType problem, int m, int iteration, double fitness, double time_ms)`
Appends a single experiment result to the CSV file.

7.17.1 Detailed Description

CSV output utilities for experiment results.

This module provides helper functions for initializing and appending experiment results to a CSV file. Output is formatted for easy analysis in external tools such as Python or spreadsheets.

7.17.2 Function Documentation

7.17.2.1 csv_algorithm_name()

```
const char * csv_algorithm_name (
    AlgorithmType alg)
```

Returns a human-readable algorithm name.

Parameters

<i>alg</i>	Algorithm type identifier.
------------	----------------------------

Returns

String representation of the algorithm.

7.17.2.2 csv_append_result()

```
int csv_append_result (
    const char * path,
    AlgorithmType alg,
    ProblemType problem,
    int m,
    int iteration,
    double fitness,
    double time_ms)
```

Appends a single experiment result to the CSV file.

Appends a single result row to the CSV file.

Parameters

<i>path</i>	Path to the CSV output file.
<i>alg</i>	Algorithm identifier.
<i>problem</i>	Problem identifier.
<i>m</i>	Problem dimension.
<i>iteration</i>	Iteration or restart index.
<i>fitness</i>	Fitness value.
<i>time_ms</i>	Runtime in milliseconds.

Returns

0 on success, 1 if path is invalid, 2 if the file cannot be opened.

7.17.2.3 csv_init_results()

```
int csv_init_results (
    const char * path)
```

Initializes a CSV results file.

Writes the CSV header row. Any existing file at the given path will be overwritten.

Parameters

<i>path</i>	Path to the CSV output file.
-------------	------------------------------

Returns

0 on success, 1 if path is invalid, 2 if the file cannot be created.

7.18 src/main.c File Reference

Main entry point for Project 3 experiments.

```
#include <stdio.h>
#include <stdlib.h>
#include "config.h"
```

```
#include "mt19937ar.h"
#include "problem.h"
#include "algorithms.h"
#include "csv.h"
```

Functions

- void [print_usage](#) (const char *prog_name)
Prints usage information for the program. Prints usage information for the program.
- int [main](#) (int argc, char **argv)
Program entry point.

7.18.1 Detailed Description

Main entry point for Project 3 experiments.

This program loads a configuration file, initializes the random number generator, executes the selected optimization algorithm, and writes per-iteration results to a CSV file.

7.18.2 Function Documentation

7.18.2.1 main()

```
int main (
    int argc,
    char ** argv)
```

Program entry point.

The program performs the following steps:

- Loads configuration values from file
- Initializes the random number generator
- Creates the selected optimization problem
- Runs the chosen algorithm
- Writes results to a CSV file

Parameters

<i>argc</i>	Argument count.
<i>argv</i>	Argument vector.

Returns

Exit status code (0 on success).

7.18.2.2 print_usage()

```
void print_usage (
    const char * prog_name)
```

Prints usage information for the program. Prints usage information for the program.

Parameters

<i>prog_name</i>	Name of the executable (typically argv[0]).
------------------	---

7.19 src/mt19937ar.c File Reference

Mersenne Twister MT19937 random number generator.

```
#include "mt19937ar.h"
```

Macros

- `#define N 624`
- `#define M 397`
- `#define MATRIX_A 0x9908b0dfUL`
- `#define UPPER_MASK 0x80000000UL`
- `#define LOWER_MASK 0x7fffffffUL`

Functions

- void `init_genrand` (uint32_t s)
Initializes the generator with a seed.
- uint32_t `genrand_int32` (void)
Generates a 32-bit unsigned random integer.
- double `genrand_real2` (void)
Generates a floating-point random number in [0, 1).

7.19.1 Detailed Description

Mersenne Twister MT19937 random number generator.

This file provides an implementation of the MT19937 pseudorandom number generator. It supports initialization with a seed and generation of 32-bit integers and double-precision floating-point values in the range [0,1).

Based on the original MT19937 reference implementation.

7.19.2 Function Documentation

7.19.2.1 genrand_int32()

```
uint32_t genrand_int32 (  
    void )
```

Generates a 32-bit unsigned random integer.

If the generator state has been exhausted, the state is regenerated automatically.

Returns

Random 32-bit unsigned integer.

7.19.2.2 genrand_real2()

```
double genrand_real2 (  
    void )
```

Generates a floating-point random number in [0, 1).

Generates a floating-point random number in the range [0, 1).

Uses a 32-bit integer random value scaled to the unit interval.

Returns

Random double in the range [0,1).

7.19.2.3 init_genrand()

```
void init_genrand (
    uint32_t s)
```

Initializes the generator with a seed.

Initializes the random number generator with a seed.

This function must be called before using the random number generator unless a default seed is acceptable.

Parameters

<code>s</code>	Seed value.
----------------	-------------

7.20 src/population.c File Reference

[Population](#) and fitness management utilities.

```
#include "population.h"
#include "mt19937ar.h"
#include <stdlib.h>
```

Functions

- int [population_init](#) ([Population](#) *pop, int n, int m)
Initializes a population structure.
- void [population_free](#) ([Population](#) *pop)
Frees memory associated with a population.
- int [fitness_init](#) ([Fitness](#) *fit, int n)
Initializes a fitness structure.
- void [fitness_free](#) ([Fitness](#) *fit)
Frees memory associated with a fitness structure.
- const double * [population_row](#) (const [Population](#) *pop, int i)
Returns a pointer to a specific row in the population.
- void [population_randomize](#) ([Population](#) *pop, double lower, double upper)
Randomizes all individuals in the population.
- void [population_evaluate](#) (const [Population](#) *pop, const [Problem](#) *prob, [Fitness](#) *fit)
Evaluates the fitness of each individual in the population.

7.20.1 Detailed Description

[Population](#) and fitness management utilities.

This module provides helper functions for initializing, randomizing, evaluating, and freeing populations and associated fitness values used in optimization experiments.

7.20.2 Function Documentation

7.20.2.1 fitness_free()

```
void fitness_free (
    Fitness * fit)
```

Frees memory associated with a fitness structure.

Parameters

<code>fit</code>	Pointer to Fitness structure.
------------------	---

7.20.2.2 fitness_init()

```
int fitness_init (
    Fitness * fit,
    int n)
```

Initializes a fitness structure.

Parameters

<i>fit</i>	Pointer to Fitness structure.
<i>n</i>	Number of fitness values.

Returns

0 on success, 1 on invalid arguments, 2 on allocation failure.

7.20.2.3 population_evaluate()

```
void population_evaluate (
    const Population * pop,
    const Problem * prob,
    Fitness * fit)
```

Evaluates the fitness of each individual in the population.
[Fitness](#) values are stored in the corresponding [Fitness](#) structure.

Parameters

<i>pop</i>	Pointer to Population structure.
<i>prob</i>	Pointer to the optimization problem.
<i>fit</i>	Pointer to Fitness structure.

7.20.2.4 population_free()

```
void population_free (
    Population * pop)
```

Frees memory associated with a population.

Parameters

<i>pop</i>	Pointer to Population structure.
------------	--

7.20.2.5 population_init()

```
int population_init (
    Population * pop,
    int n,
    int m)
```

Initializes a population structure.

Allocates memory for an n-by-m matrix where each row represents a candidate solution vector.

Parameters

<i>pop</i>	Pointer to Population structure.
<i>n</i>	Number of individuals.
<i>m</i>	Dimension of each individual.

Returns

0 on success, 1 on invalid arguments, 2 on allocation failure.

7.20.2.6 population_randomize()

```
void population_randomize (
    Population * pop,
    double lower,
    double upper)
```

Randomizes all individuals in the population.

Fills a population with uniformly random values.

Each element of the population is assigned a random value uniformly distributed within the given bounds.

Parameters

<i>pop</i>	Pointer to Population structure.
<i>lower</i>	Lower bound for values.
<i>upper</i>	Upper bound for values.

7.20.2.7 population_row()

```
const double * population_row (
    const Population * pop,
    int i)
```

Returns a pointer to a specific row in the population.

Returns a pointer to a specific population row.

Parameters

<i>pop</i>	Pointer to Population structure.
<i>i</i>	Row index.

Returns

Pointer to the requested population row.

7.21 src/problem.c File Reference

Benchmark optimization problem definitions and evaluation.

```
#include "problem.h"
```

```
#include <math.h>
```

Macros

- `#define M_PI 3.14159265358979323846`

Functions

- [Problem](#) [problem_create](#) ([ProblemType](#) t)
Creates a [Problem](#) structure for a given problem type.
- `const char *` [problem_name](#) (`const Problem *p`)
Returns a human-readable name for a problem.
- `double` [problem_eval](#) (`const Problem *p`, `const double *x`, `int m`)
Evaluates an optimization problem for a given solution vector.

7.21.1 Detailed Description

Benchmark optimization problem definitions and evaluation.

This module defines a collection of standard benchmark optimization problems and provides a unified interface for evaluating them.

7.21.2 Function Documentation

7.21.2.1 `problem_create()`

```
Problem problem_create (
    ProblemType t)
```

Creates a `Problem` structure for a given problem type.

Creates a `Problem` structure.

Parameters

<code>t</code>	<code>Problem</code> type identifier.
----------------	---------------------------------------

Returns

Initialized `Problem` structure.

7.21.2.2 `problem_eval()`

```
double problem_eval (
    const Problem * p,
    const double * x,
    int m)
```

Evaluates an optimization problem for a given solution vector.

Evaluates the objective function for a given problem.

Parameters

<code>p</code>	Pointer to the problem definition.
<code>x</code>	Input solution vector.
<code>m</code>	Dimension of the solution vector.

Returns

`Fitness` value, or NAN if inputs are invalid.

7.21.2.3 `problem_name()`

```
const char * problem_name (
    const Problem * p)
```

Returns a human-readable name for a problem.

Parameters

<code>p</code>	Pointer to the problem.
----------------	-------------------------

Returns

String name of the problem.

7.22 src/timing.c File Reference

Cross-platform wall-clock timing utilities.

```
#include "timing.h"  
#include <time.h>
```

Functions

- double [now_ms](#) (void)
Returns the current wall-clock time in milliseconds (POSIX).

7.22.1 Detailed Description

Cross-platform wall-clock timing utilities.

This module provides a portable function for obtaining the current wall-clock time in milliseconds. Platform-specific implementations are selected at compile time.

7.22.2 Function Documentation

7.22.2.1 now_ms()

```
double now_ms (  
    void )
```

Returns the current wall-clock time in milliseconds (POSIX).

Returns the current wall-clock time in milliseconds.

Uses CLOCK_MONOTONIC when available, otherwise falls back to clock().

Returns

Current time in milliseconds.

Index

- alg
 - Config, [13](#)
- ALG_ALL
 - config.h, [25](#)
- ALG_BLIND
 - config.h, [25](#)
- ALG_DE
 - config.h, [25](#)
- ALG_LOCAL
 - config.h, [25](#)
- ALG_PSO
 - config.h, [25](#)
- ALG_RLS
 - config.h, [25](#)
- algorithms.c
 - blind_search, [38](#)
 - diff_evo, [38](#)
 - p_swarm, [39](#)
 - repeated_local_search, [40](#)
- algorithms.h
 - blind_search, [20](#)
 - diff_evo, [20](#)
 - local_search, [21](#)
 - p_swarm, [22](#)
 - repeated_local_search, [22](#)
- AlgorithmType
 - config.h, [25](#)
- blind_search
 - algorithms.c, [38](#)
 - algorithms.h, [20](#)
- c1
 - Config, [13](#)
- c2
 - Config, [14](#)
- Config, [13](#)
 - alg, [13](#)
 - c1, [13](#)
 - c2, [14](#)
 - CR, [14](#)
 - de_strategy, [14](#)
 - F, [14](#)
 - generations, [14](#)
 - lambda, [14](#)
 - lower, [14](#)
 - m, [14](#)
 - max_ls_steps, [14](#)
 - n, [14](#)
 - neighbors, [14](#)
 - output_csv, [14](#)
 - pop_size, [15](#)
 - problem_type, [15](#)
 - seed, [15](#)
 - step_frac, [15](#)
 - swarm_size, [15](#)
 - upper, [15](#)
 - w, [15](#)
- config.c
 - config_load, [41](#)
- config.h
 - ALG_ALL, [25](#)
 - ALG_BLIND, [25](#)
 - ALG_DE, [25](#)
 - ALG_LOCAL, [25](#)
 - ALG_PSO, [25](#)
 - ALG_RLS, [25](#)
 - AlgorithmType, [25](#)
 - config_load, [25](#)
- config_load
 - config.c, [41](#)
 - config.h, [25](#)
- CR
 - Config, [14](#)
- csv.c
 - csv_algorithm_name, [41](#)
 - csv_append_result, [42](#)
 - csv_init_results, [42](#)
- csv.h
 - csv_algorithm_name, [27](#)
 - csv_append_result, [27](#)
 - csv_init_results, [28](#)
- csv_algorithm_name
 - csv.c, [41](#)
 - csv.h, [27](#)
- csv_append_result
 - csv.c, [42](#)
 - csv.h, [27](#)
- csv_init_results
 - csv.c, [42](#)
 - csv.h, [28](#)
- data
 - Population, [16](#)
- de_strategy
 - Config, [14](#)
- diff_evo
 - algorithms.c, [38](#)
 - algorithms.h, [20](#)

F

- Config, 14
- Fitness, 15
 - n, 16
 - values, 16
- fitness_free
 - population.c, 45
 - population.h, 31
- fitness_init
 - population.c, 45
 - population.h, 31
- generations
 - Config, 14
- genrand_int32
 - mt19937ar.c, 44
 - mt19937ar.h, 29
- genrand_real2
 - mt19937ar.c, 44
 - mt19937ar.h, 29
- include/algorithms.h, 19, 23
- include/config.h, 24, 26
- include/csv.h, 26, 28
- include/mt19937ar.h, 29, 30
- include/population.h, 30, 33
- include/problem.h, 34, 36
- include/timing.h, 36, 37
- init_genrand
 - mt19937ar.c, 44
 - mt19937ar.h, 29
- lambda
 - Config, 14
- local_search
 - algorithms.h, 21
- lower
 - Config, 14
- m
 - Config, 14
 - Population, 16
- main
 - main.c, 43
 - run, 11
- main.c
 - main, 43
 - print_usage, 43
- max_ls_steps
 - Config, 14
- mt19937ar.c
 - genrand_int32, 44
 - genrand_real2, 44
 - init_genrand, 44
- mt19937ar.h
 - genrand_int32, 29
 - genrand_real2, 29
 - init_genrand, 29
- n
 - Config, 14
 - Fitness, 16
 - Population, 16
- neighbors
 - Config, 14
- now_ms
 - timing.c, 49
 - timing.h, 37
- output_csv
 - Config, 14
- p_swarm
 - algorithms.c, 39
 - algorithms.h, 22
- pop_size
 - Config, 15
- Population, 16
 - data, 16
 - m, 16
 - n, 16
- population.c
 - fitness_free, 45
 - fitness_init, 45
 - population_evaluate, 46
 - population_free, 46
 - population_init, 46
 - population_randomize, 47
 - population_row, 47
- population.h
 - fitness_free, 31
 - fitness_init, 31
 - population_evaluate, 31
 - population_free, 32
 - population_init, 32
 - population_randomize, 32
 - population_row, 33
- population_evaluate
 - population.c, 46
 - population.h, 31
- population_free
 - population.c, 46
 - population.h, 32
- population_init
 - population.c, 46
 - population.h, 32
- population_randomize
 - population.c, 47
 - population.h, 32
- population_row
 - population.c, 47
 - population.h, 33
- print_usage
 - main.c, 43
- PROB_ACKLEY_ONE
 - problem.h, 35
- PROB_ACKLEY_TWO
 - problem.h, 35
- PROB_DEJONG1

- problem.h, [34](#)
- PROB_EGG HOLDER
 - problem.h, [35](#)
- PROB_GRIEWANGK
 - problem.h, [34](#)
- PROB_RASTRIGIN
 - problem.h, [34](#)
- PROB_ROSENBROCK
 - problem.h, [34](#)
- PROB_SCHWEFEL
 - problem.h, [34](#)
- PROB_SINE_ENV_SINE_WAVE
 - problem.h, [35](#)
- PROB_STRETCH_V_SINE_WAVE
 - problem.h, [35](#)
- Problem, [16](#)
 - type, [17](#)
- problem.c
 - problem_create, [48](#)
 - problem_eval, [48](#)
 - problem_name, [48](#)
- problem.h
 - PROB_ACKLEY_ONE, [35](#)
 - PROB_ACKLEY_TWO, [35](#)
 - PROB_DEJONG1, [34](#)
 - PROB_EGG HOLDER, [35](#)
 - PROB_GRIEWANGK, [34](#)
 - PROB_RASTRIGIN, [34](#)
 - PROB_ROSENBROCK, [34](#)
 - PROB_SCHWEFEL, [34](#)
 - PROB_SINE_ENV_SINE_WAVE, [35](#)
 - PROB_STRETCH_V_SINE_WAVE, [35](#)
 - problem_create, [35](#)
 - problem_eval, [35](#)
 - problem_name, [36](#)
 - ProblemType, [34](#)
- problem_create
 - problem.c, [48](#)
 - problem.h, [35](#)
- problem_eval
 - problem.c, [48](#)
 - problem.h, [35](#)
- problem_name
 - problem.c, [48](#)
 - problem.h, [36](#)
- problem_type
 - Config, [15](#)
- ProblemType
 - problem.h, [34](#)
- Project 3 - Differential Evolution & Partile Swarm Algorithms (C), [1](#)
- read_cfg_lines
 - run, [11](#)
- read_run_csv
 - run, [12](#)
- repeated_local_search
 - algorithms.c, [40](#)
 - algorithms.h, [22](#)
- run, [11](#)
 - main, [11](#)
 - read_cfg_lines, [11](#)
 - read_run_csv, [12](#)
 - run_once, [12](#)
 - set_cfg_value, [12](#)
 - write_cfg, [12](#)
- run_once
 - run, [12](#)
- seed
 - Config, [15](#)
- set_cfg_value
 - run, [12](#)
- src/algorithms.c, [37](#)
- src/config.c, [40](#)
- src/csv.c, [41](#)
- src/main.c, [42](#)
- src/mt19937ar.c, [44](#)
- src/population.c, [45](#)
- src/problem.c, [47](#)
- src/timing.c, [49](#)
- step_frac
 - Config, [15](#)
- swarm_size
 - Config, [15](#)
- timing.c
 - now_ms, [49](#)
- timing.h
 - now_ms, [37](#)
- type
 - Problem, [17](#)
- upper
 - Config, [15](#)
- values
 - Fitness, [16](#)
- w
 - Config, [15](#)
- write_cfg
 - run, [12](#)